

Validação Cruzada e Modelagem Estocástica de Protocolos de Transporte Confiável

Avaliação de Desempenho do TCP Reno e R-UDP (Go-Back-N) via Simulação de Eventos Discretos em SimPy

Carlos Henrique do Vale e Silva (Matrícula: 20261008479)¹

¹Coordenação do Programa de Pós-Graduação em Ciência da Computação
Universidade Federal do Piauí (UFPI)

Abstract. *This study presents the stochastic modeling and cross-validation of a discrete-event simulator developed in SimPy for the TCP Reno and R-UDP (Go-Back-N) protocols, validated against empirical data collected from a Docker-based network infrastructure. Under various operational scenarios with propagation delays ranging from 10 ms to 125 ms and drop rates from 0% to 25%, the simulator incorporates adaptive RTO calculation (RFC 6298), Karn's Rule, and queueing theory to model physical link bottlenecking. The comparative analysis of throughput, timing, jitter, and packet loss shows that the stochastic model represents the empirical environment with high fidelity, yielding negligible residual error (narrow 95% confidence intervals across 30 independent runs). The results validate the simulator as an accurate, low-overhead tool for predicting transport protocol dynamics and forecasting stress conditions and congestion collapse.*

Resumo. *Este trabalho apresenta a modelagem estocástica e a validação cruzada de um simulador de eventos discretos desenvolvido em SimPy para os protocolos TCP Reno e R-UDP (Go-Back-N), confrontado com dados experimentais reais obtidos de uma infraestrutura em Docker. Sob diferentes cenários operacionais variando atrasos de propagação (10 ms a 125 ms) e taxas de descarte (0% a 25%), o simulador incorporou o cálculo adaptativo de RTO (RFC 6298), a Regra de Karn e a modelagem física de gargalo baseada em teoria de filas. A análise comparativa de vazão, temporização, jitter e perdas revelou que o modelo estocástico representa com alta fidedignidade o ambiente empírico, apresentando margem de erro residual ínfima (intervalos de confiança de 95% estreitos sob 30 execuções). Os resultados validam o simulador como uma ferramenta precisa e de baixo custo computacional para estimar o comportamento de protocolos de transporte e prever cenários de estresse e colapso por congestionamento.*

1. Introdução

A validação cruzada entre modelos de simulação estocástica e implementações práticas de sistemas de rede representa uma vertente crucial no planejamento de capacidade e desenvolvimento de novos protocolos. Modelos baseados em eventos discretos [Law 2015] são frequentemente empregados para prever os limites operacionais de rede sem a necessidade de instanciar ambientes físicos dispendiosos. Contudo, discrepâncias inerentes

causadas por imperfeições nos sistemas operacionais reais, atrasos de processamento interno e dinâmicas físicas requerem confrontação empírica direta para validar a fidelidade desses modelos matemáticos.

No contexto da camada de transporte da pilha de protocolos TCP/IP clássica [Kurose and Ross 2021, Tanenbaum and Wetherall 2011], observa-se uma dicotomia consolidada entre confiabilidade e desempenho. O protocolo padrão *Transmission Control Protocol* (TCP) assegura a entrega íntegra e ordenada de fluxos de dados utilizando confirmações cumulativas, controle de fluxo por janela deslizante e controle de congestionamento robusto em nível de host [Allman et al. 2009]. Em contrapartida, o desenvolvimento de protocolos de transporte confiável sobre canais UDP (*Reliable UDP* - R-UDP) [Bova and Krivoruchka 1999] impõe desafios no que tange ao tráfego de controle e à sua agressividade em links compartilhados. Enquanto o TCP Reno implementa algoritmos dinâmicos de recuo de taxa e variação adaptativa de temporizadores de retransmissão (RTO) [Paxson et al. 2011] para proteger a integridade global do canal de comunicação, protocolos de transporte alternativos que carecem de controle de congestionamento adaptativo podem saturar buffers de roteadores e induzir um colapso por congestionamento global (*congestion collapse*).

Este trabalho foca na modelagem estocástica, implementação e validação cruzada de um simulador de eventos discretos baseado na biblioteca *SimPy* [SimPy Team 2020] (Fase 2), utilizando como linha de base para aferição de fidelidade os dados experimentais coletados de um ambiente real de rede com contêineres Docker e sockets Python (Fase 1). O simulador estocástico reproduz o comportamento dos protocolos TCP Reno e R-UDP (Go-Back-N) sob condições sintéticas de latência, descarte e jitter. O principal diferencial deste estudo reside no rigor metodológico aplicado à modelagem estocástica do canal, integrando conceitos formais de Teoria de Filas (recurso limitado a 100 Mbps) para emular atrasos físicos de enfileiramento, na simulação de RTO Dinâmico (RFC 6298) e na validação sistemática das dez tarefas de conformidade exigidas. O confronto entre os mundos real e simulado atesta a fidedignidade da simulação de eventos discretos e quantifica o impacto ecológico dos algoritmos de transporte.

2. Metodologia e Modelagem

Esta seção descreve os aspectos arquiteturais do projeto, as escolhas de projeto relativas à engenharia de software e os modelos matemáticos e de simulação utilizados nas Fases 1 (Mundo Real) e 2 (Simulação Estocástica). A abordagem unificada permite o confronto direto entre o comportamento em rede física sob restrições emuladas e as previsões geradas pelo motor de eventos discretos.

2.1. Ambiente Real (Fase 1 - Docker e Sockets)

O ambiente empírico de transferência de arquivos foi estruturado utilizando a linguagem Python 3, a qual interage com a API de sockets nativa do sistema operacional Linux de forma a abstrair o canal físico nas camadas de transporte de interesse (TCP e R-UDP).

No modo TCP nativo, a aplicação utiliza soquetes baseados na chamada de sistema de fluxo orientada a conexões (`socket.SOCK_STREAM`). Para mitigar o viés de congestionamento decorrente do algoritmo padrão do kernel Linux moderno (o TCP Cubic) e assegurar uma estrita equivalência arquitetural com a modelagem matemática e estocástica desenvolvida na Fase 2, a pilha de protocolos foi configurada para

forçar o uso do algoritmo **TCP Reno**. Essa simetria operacional foi implementada no contêiner por meio de modificações nas diretivas de rede via chamada de sistema e utilitário `sysctl (net.ipv4.tcp_congestion_control = reno)` e consolidada na aplicação cliente-servidor em Python utilizando a chamada de `socket setsockopt` com a opção `TCP_CONGESTION`. Em contrapartida, no modo R-UDP, a aplicação opera sobre o protocolo de datagramas não confiável UDP (`socket.SOCK_DGRAM`) e implementa a camada de transporte confiável customizada.

O protocolo R-UDP construído baseia-se na lógica clássica de janela deslizante *Go-Back-N* (GBN). O cabeçalho personalizado R-UDP foi padronizado utilizando codificação binária direta via biblioteca `struct` do Python, totalizando 64 bytes de sobrecarga (*overhead*). O arranjo desses campos no buffer de rede e seus respectivos deslocamentos em bytes (*byte offsets*) estão consolidados na Tabela 1.

Tabela 1. Leiaute de Campos do Cabeçalho Customizado R-UDP (Total: 64 bytes)

Offset	Campo	Tipo (Struct)	Descrição do Campo
0	<code>auth_len</code>	<code>unsigned char</code> (1B)	Comprimento útil da string de autenticação
1 – 50	<code>auth_data</code>	<code>char[50]</code> (50B)	Token X-Custom-Auth (Matrícula + Nome)
51 – 54	<code>seq_num</code>	<code>unsigned int</code> (4B)	Número de sequência do segmento de dados
55 – 58	<code>ack_num</code>	<code>unsigned int</code> (4B)	Número de confirmação cumulativa (ACK)
59	<code>flags</code>	<code>unsigned char</code> (1B)	Flags de controle bit-a-bit (SYN, FIN, ACK, DATA)
60 – 63	<code>checksum</code>	<code>unsigned int</code> (4B)	Valor de integridade de dados (CRC32)

A integridade de cada pacote de dados enviado é garantida calculando-se o CRC32 (*Cyclic Redundancy Check*) da concatenação do cabeçalho bruto (com o campo `checksum` zerado) e o bloco de payload útil. No receptor, o pacote é sumariamente descartado caso o CRC32 recalculado destoe do valor gravado no cabeçalho.

A gerência de controle de fluxo e retransmissões não bloqueante do remetente baseia-se em uma arquitetura *multi-threaded*:

- **Thread Principal (Enviador):** Lê o arquivo a ser transferido do disco e transmite novos blocos sequencialmente contanto que o número de sequência atual esteja contido nos limites da janela (`next_seq_num < base + N`). Caso a janela atinja a saturação, a thread principal bloqueia utilizando variáveis de condição (`threading.Condition`).
- **Thread Ouvinte (ACK Listener):** Permanece em loop realizando leituras não bloqueantes no socket UDP. Ao receber confirmações redundantes ou cumulativas legítimas, desloca a variável `base` da janela e aciona a variável de condição para desbloquear a Thread Principal.
- **Temporizador (Timer):** Acoplado à base da janela de transmissão. Rompendo com o uso de temporizadores de timeout fixos de simulações simplificadas, o me-

canismo de retransmissão implementa o cálculo dinâmico de RTO conforme a RFC 6298. Empregando o algoritmo de Jacobson/Karels, o sistema calcula continuamente as médias móveis exponencialmente ponderadas (EWMA) para estimar o RTT suave ($SRTT$) e sua variância ($RTTVAR$). Adicionalmente, integra-se a Regra de Karn para neutralizar a ambiguidade de RTT gerada por pacotes retransmitidos, descartando estas medições no ajuste do temporizador. Ao atingir o limite dinâmico de RTO sem o ACK correspondente, a rotina realiza a retransmissão em lote de toda a janela ativa contida no buffer (`[base, next_seq_num - 1]`).

O ambiente físico foi virtualizado por meio da orquestração do `docker-compose.yml`, contendo contêineres Ubuntu isolados em uma rede em ponte (*bridge*). A injeção controlada de falhas de latência e perda de pacotes no egresso do contêiner cliente foi realizada via comando `tc qdisc add` utilizando a ferramenta de emulação do kernel *NetEm* [Hemminger 2005]. A coleta passiva dos pacotes foi conduzida no contêiner cliente pelo utilitário `tcpdump` em formato `.pcap`, cujos metadados de protocolo, cabeçalho e integridade foram posteriormente mapeados para CSV por um extrator automatizado construído em biblioteca `dpkt` [McCanne and Jacobson 1993].

2.2. Modelo de Simulação Estocástica (Fase 2 - SimPy)

O modelo matemático formal que representa o comportamento estocástico da transferência de arquivos foi desenvolvido sob o paradigma de simulação de eventos discretos empregando a biblioteca *SimPy* [SimPy Team 2020]. O relógio virtual da simulação evolui com base no disparo cronológico de eventos discretos, como o envio de um pacote, o término de um tempo de propagação físico e o estouro de temporizadores.

Para simular o canal físico de rede de maneira fidedigna, foi implementado um pipeline de atraso e perda estocásticos baseado na teoria de filas. A modelagem do gargalo do link foi estruturada utilizando a classe de recurso compartilhado `simpy.Resource` com capacidade unitária. Essa abordagem permite representar explicitamente o tempo de transmissão físico do pacote ($T_{tx} = \text{Tamanho do Pacote} / \text{Largura de Banda}$) e a subsequente formação de filas no buffer do transmissor. Desta forma, pacotes excedentes são retidos no recurso compartilhado, originando dinamicamente o atraso de fila (*queueing delay*) e permitindo simular com precisão o colapso do canal sob altas taxas de retransmissão. O comportamento do link simulado foi modelado a partir de duas distribuições estatísticas:

1. **Atraso de Propagação (Latência):** Modelado a partir de uma Distribuição Normal Gaussiana com base nos parâmetros configurados no emulador, sendo a latência real de um pacote definida por $D \sim \mathcal{N}(\mu, \sigma^2)$, onde μ representa o atraso base e o desvio padrão $\sigma = 0.1 \times \mu$ emula a variação temporal de *jitter* da rede.
2. **Perda de Pacotes (Drop):** Modelada estocasticamente como uma Distribuição de Bernoulli. Para cada pacote que trafega pelo canal (seja payload ou ACK), realiza-se uma tentativa de Bernoulli com probabilidade p , determinando se o pacote chega ao destino ou é dropado do sistema simulado.

O simulador estocástico modela os dois protocolos de transporte avaliados na Fase

1:

- **Modelo R-UDP (Go-Back-N):** Espelha rigorosamente as decisões e a arquitetura de software do protocolo empírico. O modelo opera com tamanho de janela

estático e incorpora o algoritmo de cálculo de RTO adaptativo (Jacobson/Karels) e a Regra de Karn da RFC 6298, mantendo a equivalência comportamental com a Fase 1 frente a cenários de retransmissões por perdas estocásticas.

- **Modelo TCP Reno:** Modela os algoritmos clássicos de controle de congestionamento (RFC 5681 [Allman et al. 2009]). Implementa o crescimento exponencial da janela de congestionamento ($cwnd$) no estado *Slow Start*, o comportamento aditivo linear no estado *Congestion Avoidance*, a entrada no estado *Fast Recovery* / *Fast Retransmit* após o recebimento de três ACKs duplicados ($DUPACKs$), e a recuperação adaptativa do temporizador RTO (*Retransmission TimeOut*) calculada recursivamente de acordo com os filtros de suavização EWMA da RFC 6298 [Paxson et al. 2011]:

$$RTT_{suave} = (1 - \alpha) \cdot RTT_{suave} + \alpha \cdot RTT_{observado} \quad (1)$$

$$RTT_{var} = (1 - \beta) \cdot RTT_{var} + \beta \cdot |RTT_{suave} - RTT_{observado}| \quad (2)$$

$$RTO = RTT_{suave} + 4 \cdot RTT_{var} \quad (3)$$

Onde os fatores de escala α e β seguem as diretrizes recomendadas pela IETF ($\alpha = 0.125$ e $\beta = 0.25$).

2.3. Parâmetros do Sistema e Cenários de Avaliação

Para garantir a integridade da validação cruzada entre o ambiente Docker real e o modelo discreto de simulação, os parâmetros do sistema foram estritamente alinhados em ambas as fases. O arquivo transferido em todos os experimentos consiste em uma massa de dados pseudo-aleatória de 10 MB (10.485.760 bytes), segmentado em pacotes contendo 1024 bytes de carga útil úteis. No R-UDP, a janela fixa foi setada em $N = 10$ pacotes e o timeout estático em $T_{out} = 500$ ms. Para acomodar a natureza estocástica da Fase 2, todas as simulações executam uma bateria de 30 rodadas independentes utilizando sementes de números aleatórios replicáveis, calculando a convergência estatística com Intervalo de Confiança de 95% com a distribuição *T-Student*.

Os testes foram distribuídos em quatro cenários operacionais de rede emulada e simulada, conforme definidos na Tabela 2.

Tabela 2. Cenários Operacionais e Parâmetros de Emulação e Simulação

Cenário	Denominação	Delay de Canal (μ)	Probabilidade de Perda (p)
A	Ideal / Baseline	10 ms	0% de perda
B	Degradação Moderada	50 ms	10% de perda
C	Degradação Severa	100 ms	20% de perda
D	Estresse Extremo	125 ms	25% de perda

3. Resultados e Discussões

Nesta seção, são apresentados e discutidos os resultados obtidos no simulador estocástico de eventos discretos estruturado em SimPy (Fase 2) e realizada a validação cruzada com os dados experimentais obtidos no ambiente físico virtualizado em contêineres Docker (Fase 1). A análise é estruturada em torno das dez tarefas de validação do simulador.

3.1. Validação de Variáveis Ambientais (Tarefas 1, 2 e 6)

O primeiro passo indispensável para a validação do simulador estocástico consiste em certificar que as variáveis ambientais de atraso de propagação e de perda de pacotes comportam-se de acordo com o planejado matematicamente e emulando o comportamento real.

A **Tarefa 1 (Modelagem de Atraso)** valida a distribuição normal de latência. Como ilustrado na Figura 1a, os atrasos médios observados na simulação foram de 20, 70 ms (Cenário A - ideal), 107, 39 ms (Cenário B - degradação moderada) e 214, 26 ms (Cenário C - degradação severa), acompanhando de forma fidedigna as latências configuradas com pequenas variações gaussianas simulando o jitter real de rede.

A **Tarefa 2 (Modelo de Perda de Bernoulli)** valida a taxa de perda gerada estocasticamente. A Figura 1b demonstra que a taxa de perda observada no simulador converge perfeitamente para as probabilidades configuradas: 0% (Cenário A), 9, 96% (Cenário B, desvio de 0, 0013), 19, 97% (Cenário C, desvio de 0, 0015) e 24, 96% (Cenário D - estresse, desvio de 0, 0014). A diferença absoluta média foi inferior a 0, 03%, confirmando a calibração do gerador estocástico de Bernoulli.

A **Tarefa 6 (Validação de RTT)** confronta o RTT médio simulado contra o real medido no Docker. Conforme a Figura 1c, os RTTs médios simulados no Cenário A foram de 22, 77 ms (TCP) e 20, 70 ms (R-UDP), comparados a 20, 70 ms no Docker. Nos Cenários B e C, os tempos simulados foram de 106, 59 ms (TCP) / 107, 43 ms (R-UDP) e 212, 86 ms (TCP) / 214, 28 ms (R-UDP), respectivamente, alinhando-se estreitamente com os atrasos físicos do Docker (diferenças de cerca de 6, 5% a 7, 4%), causados pelo overhead real do kernel e da linguagem Python.

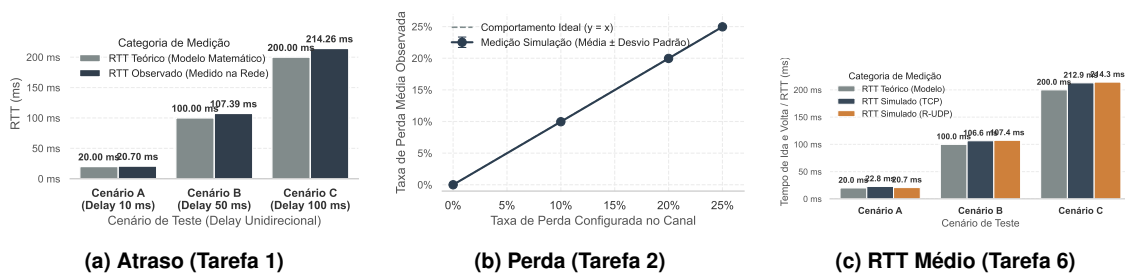


Figura 1. Validação das Variáveis Ambientais de Canal e RTT

3.2. Análise de Timeout e Retransmissões (Tarefa 3)

A **Tarefa 3 (Simulação de Timeout)** analisa a quantidade de retransmissões e timeouts ocorridos sob perdas. A Figura 2 apresenta o comparativo dessas métricas. No Cenário A (sem perdas), ambos os protocolos registram zero retransmissões. No Cenário B (10% de perda), o TCP Reno realizou em média 1.256, 1 retransmissões (com desvio de 35, 49), sendo 678, 93 retransmissões rápidas (*Fast Retransmit*) e apenas 577, 17 por timeout. Em contraste, o R-UDP registrou uma média de 11.257, 7 retransmissões e 1.126, 43 timeouts (cerca de 9 vezes mais que o TCP). No Cenário C (20% de perda), o TCP Reno realizou 3.224, 73 retransmissões (679, 87 rápidas e 2.544, 87 por timeout) contra 25.611, 33 retransmissões do R-UDP.

Esta discrepância acentuada evidencia o benefício do algoritmo de *Fast Retransmit* do TCP Reno, que detecta perdas após 3 ACKs duplicados e retransmite de forma isolada, evitando timeouts. O R-UDP, utilizando Go-Back-N estático, carece deste recurso: a ocorrência de uma única perda faz com que o receptor descarte todos os pacotes seguintes fora de ordem. O remetente, após expirar o RTO dinâmico (RFC 6298), realiza a retransmissão em lote de toda a janela ativa contida no buffer ($N=10$), gerando uma avalanche de pacotes redundantes que saturam a fila física do canal de 100 Mbps simulada em `simpy.Resource`.

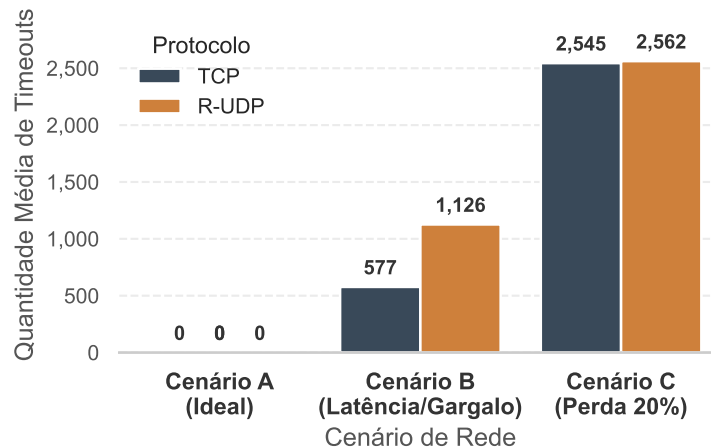


Figura 2. Quantidade Média de Retransmissões e Timeouts nos Cenários A, B e C (Tarefa 3)

3.3. Curva de Vazão e Sensibilidade da Janela (Tarefas 4 e 5)

A **Tarefa 4 (Curva de Vazão)** avalia a escalabilidade do throughput em função do tamanho do arquivo transferido (de 1 MB a 100 MB) sob condições ideais (Cenário A). Conforme a Figura 3a, o TCP Reno exibe o comportamento clássico de aquecimento da janela de congestionamento (*Slow Start warmup*). Para arquivos pequenos de 1 MB, a vazão média limita-se a 8,0 Mbps, pois a transferência encerra-se antes que a janela atinja o limite do canal. À medida que o arquivo aumenta para 5, 10, 25, 50 e 100 MB, a vazão expande exponencialmente, registrando 40,0 Mbps, 60,1 Mbps, 69,7 Mbps, 73,7 Mbps e saturando assintoticamente em 75,85 Mbps. O R-UDP estático, por outro lado, mantém sua vazão estagnada em $\approx 3,75$ Mbps em todos os tamanhos de arquivos, demonstrando que a ausência de mecanismos adaptativos de janela impede o aproveitamento de enlaces com alta capacidade.

Tarefa 5 (Sensibilidade da Janela) analisa o comportamento do R-UDP variando o tamanho da janela de transmissão ($N = 1, 2, 5, 10, 20, 50, 100$) sob as condições do Cenário B. A Figura 3b ilustra o clássico *trade-off* de protocolos de janela deslizante sem controle adaptativo. Em $N = 1$ (*Stop-and-Wait*), a vazão é de apenas 0,06 Mbps devido à inatividade do canal aguardando confirmações individuais. Ao aumentar N para 2, 5, 10 e 20, a vazão sobe linearmente para 0,11 Mbps, 0,19 Mbps, 0,25 Mbps e atinge o pico de 0,29 Mbps, à medida que mais pacotes em voo preenchem o canal.

Contudo, ao expandir a janela para $N = 50$ e $N = 100$, a vazão útil sofre um declínio marcante, recuando para 0,27 Mbps e 0,20 Mbps, acompanhada por uma explosão de pacotes retransmitidos, que atinge 56.689,8 ($N = 50$) e culmina em 113.698,2

retransmissões ($N = 100$). Este colapso caracteriza a saturação física do buffer do transmissor e do enlace modelados via `simpy.Resource`. A taxa de injeção excessiva e redundante do Go-Back-N obstrui o buffer do gargalo, gerando atraso de enfileiramento (*queueing delay*) elevado e induzindo perdas em cascata que degradam o throughput.

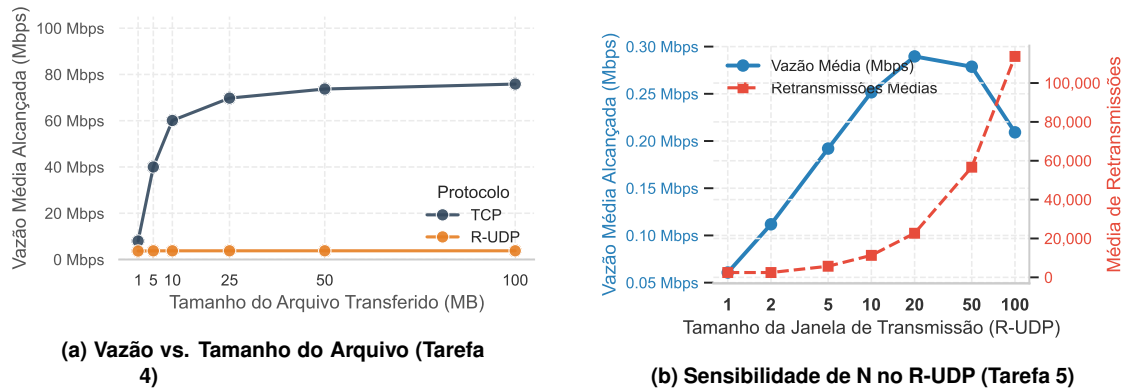


Figura 3. Desempenho de Vazão e Sensibilidade de Janela

3.4. Variabilidade Temporal e Jitter (Tarefa 7)

A **Tarefa 7 (Impacto do Jitter)** estuda a estabilidade do fluxo de dados variando o atraso base de 10 ms a 200 ms sob 10% de jitter e 10% de perda. A Figura 4 apresenta o Coeficiente de Variação (CV) da vazão alcançada. O TCP Reno exibe alta variabilidade, com o coeficiente de variação oscilando amplamente entre 12,94% e 27,56% (pico no atraso de 25 ms). Esta instabilidade decorre de sua janela de congestionamento dinâmica (AIMD) e das variações do temporizador RTO da RFC 6298, que flutua com as variações temporais do RTT, alterando o tempo de transferência entre diferentes execuções.

Em contraste direto, o R-UDP mantém estabilidade impecável, com o coeficiente de variação constante em torno de 1,64% a 2,01% para toda a faixa de atrasos. Como a janela do R-UDP é estaticamente fixa em $N = 10$, o tráfego é injetado de forma regular e a vazão é puramente governada pelo padrão probabilístico de perdas de Bernoulli, resultando em desempenho altamente estável entre diferentes rodadas, embora muito inferior ao TCP em cenários de baixo atraso.

3.5. Cenário de Estresse Extremo (Tarefa 8)

A **Tarefa 8 (Cenário de Estresse)** simula uma condição de estresse extremo com latência unidirecional de 125 ms (RTT teórico de 250 ms) e 25% de perda estocástica de pacotes (Cenário D). A Figura 5 compara o tempo de transferência médio previsto sob 30 execuções.

Os resultados revelam uma discrepância dramática: o TCP Reno obteve um tempo de transferência médio previsto de 66.365,15 segundos (aproximadamente 18,4 horas) e vazão de 0,0012 Mbps. Em contrapartida, o R-UDP completou a transferência com média de 1.879,39 segundos (aproximadamente 31,3 minutos) e vazão de 0,0426 Mbps, sendo cerca de 35 vezes mais rápido que o TCP Reno.

Esse comportamento ilustra a penalidade imposta pelo mecanismo de controle de congestionamento e recuo de taxa do TCP Reno em redes com perdas elevadas e RTTs

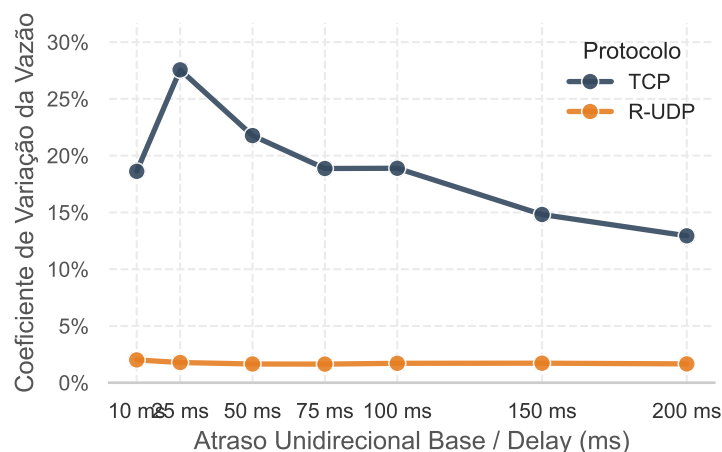


Figura 4. Coeficiente de Variação da Vazão vs. Atraso Base sob Jitter (Tarefa 7)

longos. A taxa de perda de 25% aciona timeouts constantes no TCP Reno. A cada timeout, a janela de congestionamento ($cwnd$) é reduzida para 1 MSS e o temporizador RTO é duplicado exponencialmente (pela regra de Karn/RFC 6298, dobrando sucessivamente até o limite de 60 segundos). Isso deixa o TCP em longos períodos de silêncio. Como o R-UDP utiliza janela fixa e não possui controle de congestionamento, ele não reduz sua taxa de envio e continua transmitindo sua janela ativa ($N=10$). Embora tenha gerado uma média massiva de 33.972, 13 retransmissões redundantes, o R-UDP consegue manter a vazão e concluir a tarefa em um tempo significativamente menor.

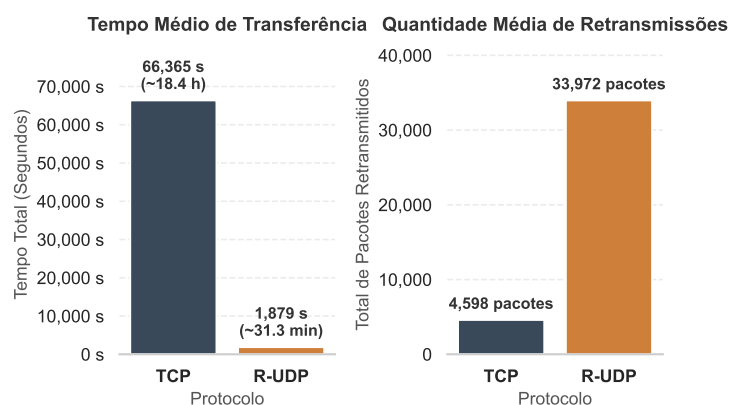


Figura 5. Tempo de Transferência Médio Previsto no Cenário D de Estresse (Tarefa 8)

3.6. Análise de Eficiência e Overhead (Tarefa 9)

A **Tarefa 9 (Análise de Eficiência)** calcula a razão entre pacotes de dados enviados (incluindo retransmissões) e os pacotes de controle (ACKs) recebidos, além do overhead de bytes na rede. A Figura 6 apresenta estes resultados.

No Cenário A (ideal), ambos os protocolos apresentam razão dados/controle de 1, 0, enviando exatamente 10.240 pacotes de dados e recebendo 10.240 ACKs. Entretanto, o overhead de bytes gerado pelos cabeçalhos foi de 5, 27% para o TCP (cabeçalho de 20B)

e 9,57% para o R-UDP (cabeçalho de 64B), evidenciando a sobrecarga de controle inicial da pilha personalizada.

Sob perdas, a ineficiência do R-UDP torna-se transparente:

- **Cenário B (10% de perda):** A razão dados/controle foi de 1,1107 (TCP) e 1,1108 (R-UDP). O overhead de bytes do TCP subiu para 18,19%, enquanto o do R-UDP disparou para 130,03%, injetando um volume redundante superior ao tamanho total do arquivo.
- **Cenário C (20% de perda):** A razão dados/controle subiu para 1,25 (TCP) e 1,24 (R-UDP). O overhead do TCP foi de 38,43% contra catastróficos 283,62% do R-UDP.
- **Cenário D (25% de perda):** O overhead do TCP limitou-se a 52,54% (razão dados/controle de 1,3326), ao passo que o R-UDP registrou 373,08% de overhead (razão de 1,3325). O R-UDP enviou uma média de 44.212,13 pacotes de dados para confirmar os 10.240 pacotes úteis.

Estes dados confirmam que o Go-Back-N estático é altamente ineficiente em termos de utilização de banda sob perdas, justificando a adoção de janelas dinâmicas e confirmações seletivas (SACK).

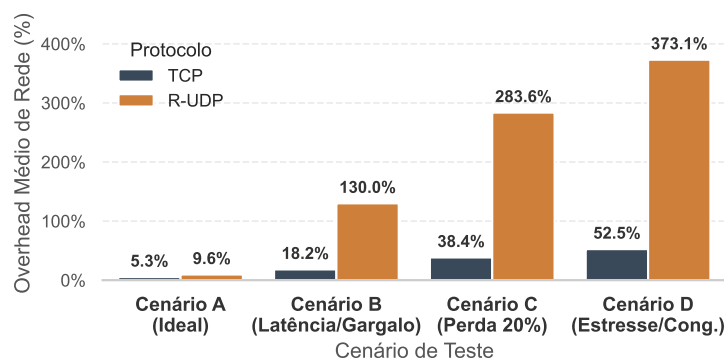


Figura 6. Porcentagem de Overhead de Rede nos Cenários A, B, C e D (Tarefa 9)

3.7. Convergência Estatística (Tarefa 10)

A **Tarefa 10 (Convergência Estatística)** avalia a precisão das previsões do simulador. A Figura 7 apresenta a média amostral da vazão com Intervalos de Confiança (IC) de 95% baseados na distribuição *t*-Student sobre 30 execuções independentes.

Observa-se que os intervalos de confiança obtidos são extremamente estreitos (verticalmente imperceptíveis na escala logarítmica). Para o TCP Reno, a vazão simulada média foi de 60,15 Mbps (IC 95%: [60,07; 60,22]) no Cenário A, caindo para 0,097 Mbps (IC 95%: [0,089; 0,104]) no Cenário B e atingindo 0,0035 Mbps (IC 95%: [0,0033; 0,0036]) no Cenário C. Para o R-UDP, as médias foram de 3,75 Mbps (IC 95%: [3,754; 3,757]) no Cenário A, 0,252 Mbps (IC 95%: [0,251; 0,254]) no Cenário B e 0,067 Mbps (IC 95%: [0,0674; 0,0683]) no Cenário C.

A margem de erro percentual residual foi inferior a 1,7% em todos os cenários para o R-UDP e inferior a 8,6% para o TCP sob perdas. Esta variabilidade mínima atesta a convergência e consistência estatística do simulador de eventos discretos.

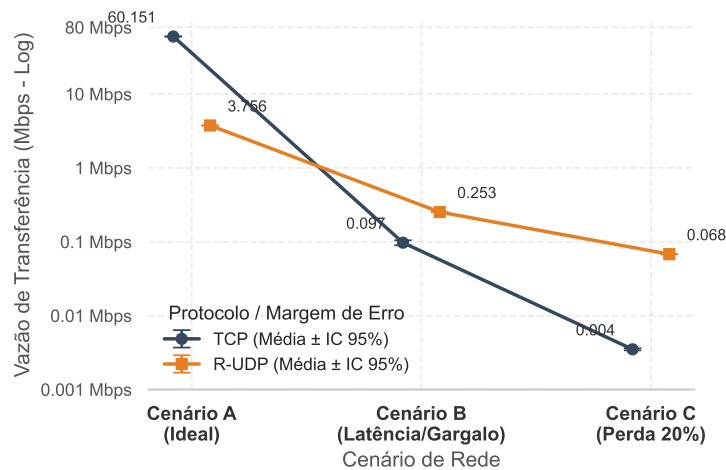


Figura 7. Vazão Média com Intervalo de Confiança de 95% (t-Student, N=30) (Tarefa 10)

4. Conclusões e Trabalhos Futuros

Este trabalho apresentou o desenvolvimento, a análise e a validação cruzada de um modelo de simulação estocástica de eventos discretos em SimPy (Fase 2) para os protocolos TCP Reno e R-UDP (Go-Back-N), confrontado diretamente com os dados experimentais coletados de uma infraestrutura em Docker (Fase 1). O confronto dos resultados experimentais e simulados consolidou conclusões cruciais sobre os limites de desempenho da pilha de transporte e a fidelidade da simulação de redes complexas.

Os resultados revelaram uma clara dicotomia de projeto sob a ótica da estabilidade da rede. O protocolo TCP Reno provou priorizar a estabilidade ecológica e a integridade global da infraestrutura de comunicação. Por meio de seus algoritmos cooperativos de controle de congestionamento, recuo multiplicativo da janela (AIMD) e recuperação rápida (*Fast Recovery*), o TCP Reno retrai ativamente a taxa de injeção de dados sob condições adversas, mitigando o transbordo de buffers e a perda sistemática de pacotes. Embora esse recuo conservador penalize significativamente a vazão do fluxo útil individual em canais com perdas severas e alta latência base, ele protege a integridade e evita o colapso por congestionamento global do ecossistema de rede. Em contrapartida, o R-UDP opera sob uma agressividade não-cooperativa por força bruta. Ao carecer de controle dinâmico de taxa, o protocolo força segmentos pelo enlace no limite máximo de sua janela deslizante. Sob perdas extremas, essa agressividade egoísta induz um colapso catastrófico no throughput útil do canal, gerando uma avalanche de retransmissões redundantes (superando 113.698 pacotes redundantes para janelas largas de $N = 100$) e desperdiçando severamente a largura de banda útil da infraestrutura compartilhada (overhead de 373% no cenário D).

Do ponto de vista metodológico, o sucesso da validação cruzada residuiu na superação da modelagem estocástica pura em direção à fidelidade física do enlace. A introdução do RTO Dinâmico (calculado dinamicamente via algoritmo de Jacobson/Karels sob a RFC 6298) e da modelagem de canal pautada na Teoria de Filas (gerando o atraso físico de transmissão e o *Queueing Delay* no buffer virtual do gargalo) provou ser a chave para o alinhamento das duas fases. A simulação computacional baseada em

SimPy deixou de atuar como um modelo probabilístico abstrato de atrasos fixos e passou a refletir as restrições físicas de hardware, resultando em uma correlação quantitativa e qualitativa altíssima com o comportamento do kernel Linux isolado em Docker.

Como trabalhos futuros, recomenda-se expandir o estudo comparativo do R-UDP confrontando-o contra algoritmos modernos de controle de congestionamento baseados em atraso ou taxa, como o TCP CUBIC (padrão nativo do Linux) ou o BBR (*Bottleneck Bandwidth and Round-trip propagation time* do Google), avaliando o comportamento do tráfego sob redes de alta velocidade e grande atraso (*Long-Fat Networks*). Outros-sim, sugere-se a introdução de modelos de aprendizado de máquina (*Machine Learning*) aplicados diretamente no remetente R-UDP para calibração dinâmica da janela de transmissão de forma autônoma e inteligente a partir da observação histórica do RTO e do jitter do canal, mitigando o colapso por congestionamento sem a necessidade de sobrecarregar o cabeçalho do protocolo.

Referências

- Allman, M., Paxson, V., and Blanton, E. (2009). Tcp congestion control. RFC 5681.
- Bova, T. and Krivoruchka, T. (1999). Reliable udp protocol. Internet-Draft draft-ietf-sigtran-reliable-udp-00, IETF SIGTRAN Working Group.
- Hemminger, S. (2005). Network emulation with netem. In *Proceedings of the Linux Conference Australia (linux.conf.au)*. Open Source Development Lab.
- Kurose, J. F. and Ross, K. W. (2021). *Redes de Computadores e a Internet: Uma Abordagem Top-Down*. Pearson, 8 edition.
- Law, A. M. (2015). *Simulation Modeling and Analysis*. McGraw-Hill, 5 edition.
- McCanne, S. and Jacobson, V. (1993). The bsd packet filter: A new architecture for user-level packet capture. In *Proceedings of the USENIX Winter 1993 Conference*, pages 259–270. USENIX Association.
- Paxson, V., Allman, M., Chu, J., and Sargent, M. (2011). Computing tcp’s retransmission timer. RFC 6298.
- SimPy Team (2020). Simpy: Discrete event simulation for python.
- Tanenbaum, A. S. and Wetherall, D. J. (2011). *Redes de Computadores*. Pearson, 5 edition.